

Actividad en Clase – Sistemas Distribuidos con RabbitMQ

Repositorio en GitHub:

https://github.com/dogor123/programaciondistribuida/tree/main/clase9/act_clase

RabbitMQ ejecutándose:

```
(venv) root@DESKTOP-E7NLUAM:/home/clase9/act_clase# service rabbitmq-server status
● rabbitmq-server.service - RabbitMQ Messaging Server
   Loaded: loaded (/usr/lib/systemd/system/rabbitmq-server.service; enabled; pre>
   Active: active (running) since Wed 2026-04-22 22:52:38 -05; 8min ago
     Main PID: 4376 (beam.smp)
        Tasks: 36 (limit: 10680)
      Memory: 121.4M (peak: 153.9M)
         CPU: 22.385s
       CGroup: /system.slice/rabbitmq-server.service
               └─4376 /usr/lib/erlang/erts-13.2.2.5/bin/beam.smp -W w -MBas ageffcbf>
                 └─4386 erl_child_setup 65536
                   └─4462 /usr/lib/erlang/erts-13.2.2.5/bin/inet_gethost 4
                     └─4463 /usr/lib/erlang/erts-13.2.2.5/bin/inet_gethost 4
                       └─4488 /bin/sh -s rabbit_disk_monitor

Apr 22 22:52:27 DESKTOP-E7NLUAM systemd[1]: Starting rabbitmq-server.service - Rab>
Apr 22 22:52:38 DESKTOP-E7NLUAM systemd[1]: Started rabbitmq-server.service - Rab>
lines 1-16/16 (END)
```

En esta captura se verifica que el servicio RabbitMQ Messaging Server está activo y en ejecución dentro del entorno WSL. El comando "service rabbitmq-server status" confirma el estado "active (running)", lo que indica que el broker de mensajes está listo para recibir y distribuir mensajes entre los servicios productores y consumidores. Este es el primer paso indispensable antes de ejecutar cualquier componente del sistema distribuido, ya que RabbitMQ actúa como intermediario central de comunicación.

Creación del archivo producer.py:

```
GNU nano 7.2 producer.py
import pika

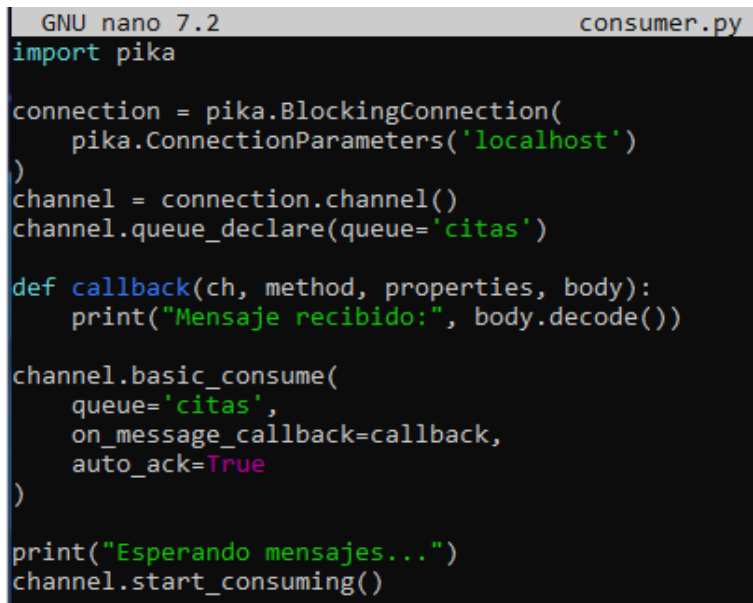
connection = pika.BlockingConnection(
    pika.ConnectionParameters('localhost')
)
channel = connection.channel()
channel.queue_declare(queue='citas')

channel.basic_publish(
    exchange='',
    routing_key='citas',
    body='Nueva cita creada'
)

print("Mensaje enviado a la cola")
connection.close()
```

Se muestra el contenido del archivo producer.py editado con GNU nano. Este script implementa el rol de productor dentro del patrón de comunicación asíncrona: establece una conexión con RabbitMQ en localhost, declara la cola "citas" y publica el mensaje "Nueva cita creada" usando el método basic_publish. Al finalizar cierra la conexión correctamente. Este componente representa el servicio que genera eventos o solicitudes sin necesidad de esperar una respuesta inmediata, lo que elimina el acoplamiento directo con el consumidor.

Creación del archivo consumer.py:



```
GNU nano 7.2 consumer.py
import pika

connection = pika.BlockingConnection(
    pika.ConnectionParameters('localhost')
)
channel = connection.channel()
channel.queue_declare(queue='citas')

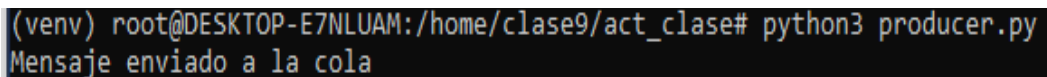
def callback(ch, method, properties, body):
    print("Mensaje recibido:", body.decode())

channel.basic_consume(
    queue='citas',
    on_message_callback=callback,
    auto_ack=True
)

print("Esperando mensajes...")
channel.start_consuming()
```

En esta imagen se aprecia el archivo consumer.py, también editado en GNU nano. Este script actúa como consumidor: se conecta a RabbitMQ, declara la misma cola "citas" para asegurar su existencia, define una función callback que imprime cada mensaje recibido y se suscribe a la cola mediante basic_consume con auto_ack activado. Finalmente, el método start_consuming mantiene el proceso activo escuchando de forma continua. Este enfoque permite procesar mensajes de manera asíncrona a medida que llegan, sin bloquear otros procesos del sistema.

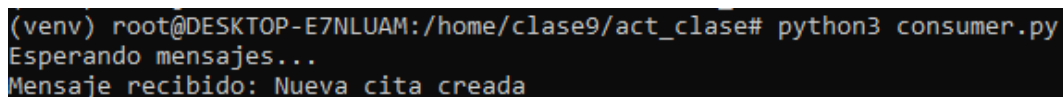
Envío del mensaje – Ejecución del producer:



```
(venv) root@DESKTOP-E7NLUAM:/home/clase9/act_clase# python3 producer.py
Mensaje enviado a la cola
```

Esta captura muestra la ejecución exitosa del producer.py mediante el comando "python3 producer.py". El mensaje de confirmación "Mensaje enviado a la cola" indica que el script logró conectarse a RabbitMQ, publicar el mensaje en la cola "citas" y cerrar la conexión sin errores. Esto valida que el productor funciona correctamente y que el broker está disponible y operativo para recibir mensajes.

Recepción del mensaje – Ejecución del consumer:

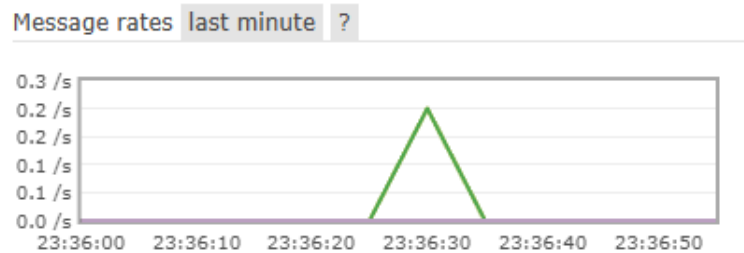


```
(venv) root@DESKTOP-E7NLUAM:/home/clase9/act_clase# python3 consumer.py
Esperando mensajes...
Mensaje recibido: Nueva cita creada
```

Aquí se evidencia que el consumer.py recibió y procesó correctamente el mensaje publicado por el producer. Al ejecutar "python3 consumer.py", el proceso quedó en escucha activa con el mensaje

"Esperando mensajes...", y al llegar el mensaje desde la cola mostró "Mensaje recibido: Nueva cita creada". Esto confirma el funcionamiento completo del patrón productor-cola-consumidor, demostrando que la comunicación asíncrona entre servicios desacoplados opera de forma correcta.

Panel web de RabbitMQ – Tasa de mensajes:



La gráfica del panel de administración web de RabbitMQ (accesible en localhost:15672) muestra la tasa de mensajes procesados durante el último minuto. El pico visible en la curva verde corresponde exactamente al momento en que se ejecutó el producer y el consumer, evidenciando el flujo de publicación y consumo de mensajes en tiempo real. Este panel es una herramienta fundamental para monitorear el comportamiento del broker, identificar cuellos de botella y verificar que los mensajes están siendo entregados correctamente.